Introduction to Computer Science: Exam Companion

Innleiðing í teldufrøði (005010.26)
Fall 2026

Oddmar Dam — Version 1.0.0, October 13, 2026

The exam in one paragraph

Wednesday 21 October 2026, 10:00–14:00, in the Aula at Frælsið (Moodle, 13 Oct). 4 hours, written, no electronics. Allowed: the textbook (Brookshear & Brylow), notes handed out, *my own notes* and the two required submissions. The old test exams are built from 8 sections of about 10–14 points each, with a mix of multiple choice, short explanations (*"use between 20–50 words"*), tracing a program or a data structure, and "write this in Python / SQL / machine code". Read the *points* per question: a 2-point question wants one line, a 6-point one wants working shown.

Question map: where each old question type is covered

Old exam section	Topic	This document
Test 1 §1, Test 2 §1–2	bits, two's complement, float, parity, Hamming, truth tables	§1 Data representation
Test 1 §2, Test 2 §4.4	Vole machine code	§2 Machine language
Test 1 §3, Test 2 §1.2	OS, networks, protocols	§3 Operating systems and networks
Test 1 §4–5, Test 2 §2.2–2.4	loops, search, Python, translation, scope	§4 Algorithms and Python, §5 Languages
Test 1 §6, Test 2 §4.3	lifecycle, UML class diagrams	§6 Software engineering
Test 1 §7, Test 2 §3–4	lists, trees, pointers in memory, traversals	§7 Data structures
Test 1 §8, Test 2 §5.1	ER, relational algebra, SQL	§8 Databases

Contents

1 Data representation (Brookshear ch. 1) 3
1.1 Hexadecimal, binary, unsigned . . . 3
1.2 Two's complement . . . 3
1.3 Excess notation . . . 3
1.4 Fixed point and the 8-bit floating-point format . . . 3
1.5 Bit operations and gates . . . 4
1.6 Parity and error-correcting codes . . . 4
1.7 Storage arithmetic . . . 5

2 Machine language: the Vole machine (Brookshear ch. 2 and Appendix C) 6
2.1 Writing programs . . . 7

3 Operating systems and networks (Brookshear ch. 3 and 4) 8
3.1 The 20–50 word answers . . . 8
3.2 Networks cheat sheet . . . 8

4 Algorithms and Python (Brookshear ch. 5, practice in Python) 10
4.1 Tracing loops . . . 10
4.2 Search . . . 10

4.3 Writing the loops (templates) . . . 10
4.4 More Python from the labs (Moodle labs 5a, 5b, 6b and the array/function notes) . . . 11
4.5 Recursion . . . 12

5 Programming languages (Brookshear ch. 6) 12
5.1 Syntax diagrams and parse trees . . . 12

6 Software engineering and UML class diagrams (Brookshear ch. 7) 13
6.1 Class diagrams and multiplicity . . . 13

7 Data structures in memory (Brookshear ch. 8) 14
7.1 Linked lists in memory . . . 14
7.2 Binary trees in memory . . . 14
7.3 Tree vocabulary and traversals . . . 14
7.4 Stacks, queues and how structures are stored (book ch. 8, slides) . . . 15
7.5 Arrays . . . 16

8 Databases (Brookshear ch. 9 and the ER notes) 17
8.1 From tables to an ER diagram . . . 17
8.2 Database vocabulary (book 9.1–9.2 and the ER notes) . . . 17
8.3 Relational algebra (Brookshear's notation) . . . 18
8.4 SQL . . . 18

9 The rest of the syllabus (slides and book, not in the old exams) 19
9.1 Data storage (ch. 1) . . . 19
9.2 CPU, machine cycle and I/O (ch. 2) . . . 19
9.3 Operating systems (ch. 3) . . . 20
9.4 Programming languages and implementation (ch. 6) . . . 20
9.5 Algorithms: concepts (ch. 5) . . . 20
9.6 Software engineering: the rest of 7.1–7.5 . . . 21

10 Chapter review problems from Moodle, worked (ch. 6–9) 22

11 Last-minute checklist 25

1 Data representation (Brookshear ch. 1)

1.1 Hexadecimal, binary, unsigned

hex	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
bin	0000	0001	0010	0011	0100	0101	0110	0111	1000	1001	1010	1011	1100	1101	1110	1111
dec	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15

One hex digit is exactly four bits, so conversion is digit by digit. Unsigned value $= \sum$ (bit $\times$ position value $128, 64, 32, 16, 8, 4, 2, 1$). **Test 2 §1.4:** 11010011 = `D3` $= 128 + 64 + 16 + 2 + 1 = 211$. `FF` is 11111111, and read as two's complement it is -1. n hex digits of address means 16^n cells: 2 digits $= 256$, 4 digits $= 65\,536$.

1.2 Two's complement

Method: reading an 8-bit pattern as two's complement

1. Leading bit 0: positive, read it as unsigned.
2. Leading bit 1: negative. **Invert all bits, add 1, read the result and put a minus sign in front** (or: unsigned value -256).
3. Range with n bits: $-2^{n-1} \ldots 2^{n-1} - 1$ (4 bits: $-8 \ldots 7$, 8 bits: $-128 \ldots 127$).
4. **Hex shortcut:** first hex digit `0`–`7` means non-negative, `8`–`F` negative. (Test 1 §1.2: `6A` and `21` are non-negative; `D4` and `EF` are not.)

Example: 10011100 unsigned $= 128+16+8+4 = 156$. As two's complement: invert $\to$ 01100011, add 1 $\to$ 01100100 $= 100$, so -100 (check: $156 - 256 = -100$).

Subtraction = add the negative. Test 1 §1.3: $1 - 4$ in 4 bits: $-4 = 1100$ (invert 0100 $\to$ 1011, add 1 $\to$ 1100), so $0001 + 1100 = 1101 = -3$.

Overflow in two's complement addition

Overflow happens *only* when you add two numbers of the **same sign** and the result has the **opposite sign**. Positive + negative can never overflow. 4-bit examples: $0100 + 0100 = 1000$ ($4+4$ gives -8) **overflow**; $1001 + 1111 = 1000$ ($-7 + -1 = -8$) fine; $1000 + 1111 = 0111$ ($-8 + -1$ gives $+7$) **overflow**; $0011 + 0100 = 0111$ ($3 + 4 = 7$) fine.

1.3 Excess notation

k-bit excess notation stores value + bias, with bias 2^{k-1}. So **value = pattern (as unsigned) − bias**. 3 bits, excess-4: 000 $= -4$, 001 $= -3$, 010 $= -2$, 011 $= -1$, **100 = 0**, 101 $= 1$, 110 $= 2$, 111 $= 3$. (Test 2 §2.1B: "which 3-bit pattern is 0 in excess notation": 100.) 6 bits, excess-32: $011111 = 31 - 32 = -1$, $100110 = 38 - 32 = 6$, $111000 = 24$, $000101 = -27$, $010101 = -11$.

1.4 Fixed point and the 8-bit floating-point format

Binary fractions: $.1 = \frac{1}{2}$, $.01 = \frac{1}{4}$, $.001 = \frac{1}{8}$, $.0001 = \frac{1}{16}$. Fixed point (4+4 bits): `0101.1010` $= 5 + \frac{1}{2} + \frac{1}{8} = 5.625$.

The course's 8-bit float: sign (1) | exponent (3, excess-4) | mantissa (4)

value $= (-1)^{\text{sign}} \times 0.\text{mmmm}_2 \times 2^{e-4}$ where e is the exponent field as unsigned. **Decode:** sign; exponent field -4; mantissa $= .mmmm$ (a leading 0.); move the point that many places (right if positive). **Encode:** write the number in binary, move the point to the left of the leading 1 ($\to$ mantissa $.1xxx$, 4 bits, *truncate* the rest) and count the places moved $= k$; exponent field $= k + 4$.

- **Decode** 11001010 (Test 1 §1.2): sign 1, exponent $100 = 4 \to 0$, mantissa $.1010 = \frac{5}{8}$, so $-\frac{5}{8}$ (answer C).
- **Decode** 01011010: sign 0, exponent $101 = 5 \to +1$, mantissa $.1010 = \frac{5}{8}$, so $\frac{5}{8} \cdot 2^1 = \frac{5}{4} = 1.25$.

- **Encode** $6.75 = 110.11_2 = .11011 \times 2^3$: mantissa truncated to .1101, exponent $3+4=7=111$, sign 0: `0 111 1101` = `7D`, which decodes to $.1101 \times 2^3 = 6.5$ (truncation lost the $\frac{1}{4}$). A value that fits, $3.25 = 11.01_2 = .1101 \times 2^2$: `0 110 1101` and no loss.
- **Encode** $\frac{15}{16} = .1111_2$, exponent $0+4=100$: `0 100 1111` = `4F` (Test 2 §2.1).
- Which pattern *is* $\frac{15}{16}$ in fixed point *xxxx.xxxx*: 0000.1111 (Test 2 §2.1A).

1.5 Bit operations and gates

Bitwise, column by column. **AND**: 1 only if both are 1; **OR**: 0 only if both are 0; **XOR**: 1 if the bits differ.

```
10110011 AND 01111010 = 00110010   10110011 OR 01111010 = 11111011   10110011 XOR 01111010 = 11001001
```

Method for "what does this circuit compute?" Write the four rows $AB = 00, 01, 10, 11$, compute the output of each gate in turn (one column per gate), and compare the last column with the known tables.

A	B	AND	OR	NAND	NOR	XOR	$\neg A \wedge B$
0	0	0	0	1	1	0	0
0	1	0	1	1	0	1	1
1	0	0	1	1	0	1	0
1	1	1	1	0	0	0	0

The last column is a circuit made of a NOT on A feeding an AND with B: it matches *none* of AND, OR, NAND, XOR, so the answer would be "none of the above" (it is "not A and B"). Test 1 §1.1 and Test 2 §1.1 expect $0, 1, 1, 0 =$ **XOR**.

1.6 Parity and error-correcting codes

Parity

Even parity: every pattern must contain an *even* number of 1s; **odd parity**: an odd number. A pattern with the wrong count **definitely** has an error (an even number of flipped bits would go unnoticed, so a correct count is only "no error detected").

- Test 1 §1.5, even parity: 101110101 (six 1s) fine; 111110000 (five) **error**; 100010010 (three) **error**; 000000000 fine. Answer B, C.
- Test 2 §1.3, even parity: 011110000 (4) ok, 101111100 (6) ok, 000101111 (5) **error**, 111111010 (7) **error**. Answer c, d.
- Odd parity: 101100101 (5) ok, 111000111 (6) **error**, 000000001 (1) ok, 011011011 (6) **error**.

Method: decoding with an error-correcting code (Hamming distance)

The **Hamming distance** of two patterns is the number of positions where they differ. For each received word compute the distance to every code word (XOR and count the 1s) and choose the **nearest**. Distance 0 means received correctly, 1 means one bit was corrected.

Example (Test 2 §1.3.2). Codes: A= 000000, B= 001111, C= 010011, D= 011100, E= 100110, F= 101001, G= 110101, H= 111010. Received 101101: distances to A..H are $4, 2, 5, 3, 3, \mathbf{1}, 2, 4$, so F. Doing all six words:

received	101101	010000	011011	000010	010100	100010
nearest (distance)	F (1)	A (1)	C (1)	A (1)	D (1)	E (1)

The word is **F A C A D E** ("FACADE").

1.7 Storage arithmetic

- **Disk capacity** = surfaces × tracks × sectors × bytes per sector (5 platters = 10 surfaces, 10 tracks, 8 sectors, 512 B: $10 \cdot 10 \cdot 8 \cdot 512 = 409\,600$ B).
- **Images:** bits per image = rows × columns × bits per pixel; photos that fit = capacity in bits / bits per image (500 MB, 512×512 at 1 bit: $\frac{500 \cdot 2^{20} \cdot 8}{512^2} = 16\,000$).
- Binary prefixes: 1 KiB $= 2^{10}$, 1 MiB $= 2^{20}$, 1 GiB $= 2^{30}$ bytes.

2 Machine language: the Vole machine (Brookshear ch. 2 and Appendix C)

16 registers `0`–`F` (1 byte each), 256 memory cells (1 byte each), every instruction is **two bytes** (4 hex digits = op, then three operand digits). Instructions sit at **even** addresses (`00, 02, 04, ...`).

Code	Meaning	Example
`1RXY`	LOAD register R from memory cell XY	`1A1A`: RA ← mem[1A]
`2RXY`	LOAD register R with the *value* XY	`220C`: R2 ← 0C
`3RXY`	STORE register R into cell XY	`30A0`: mem[A0] ← R0
`40RS`	MOVE register R to register S	
`5RST`	ADD $S + T$ as two's complement into R	`5423`: R4 ← R2+R3
`6RST`	ADD $S + T$ as floating point into R	
`7RST`	$R \leftarrow S$ OR T	`70A2`: R0 ← RA OR R2
`8RST`	$R \leftarrow S$ AND T	
`9RST`	$R \leftarrow S$ XOR T	
`AR0X`	ROTATE register R right X times	
`BRXY`	JUMP to XY **if** $R =$ R0	`B130`: if R1 = R0 go to 30; `B0XY` always jumps
`C000`	HALT	

Mixing up `1` and `2` (memory vs. immediate value) and `3` (the destination is memory) are the usual mistakes.

Method: tracing a program

1. Write the registers as a small table, all unknown ones = `00`.
2. Start at the given PC. One instruction at a time: split into op and operands, do it, update the table. After each fetch the PC has moved on by 2.
3. A jump replaces the PC; if the condition is false execution just continues.
4. **Count executed instructions including the HALT.**
5. After HALT, **PC = address of the HALT** +2 and **IR** = `C000`.

Test 1 §2 (PC starts at `1A`; cells `1A`:`220C`, `1C`:`1A1A`, `1E`:`70A2`, `20`:`212E`, `22`:`B130`, `24`:`A248`, `26`:`BCDE`, `28`:`C000`, `30`:`C000`):

addr	instruction	effect
1A	`220C`	R2 = `0C`
1C	`1A1A`	RA = contents of cell 1A = `22` (the cell holds the first byte of `220C`)
1E	`70A2`	R0 = RA OR R2 = 00100010 ∨ 00001100 = 00101110 = `2E`
20	`212E`	R1 = `2E`
22	`B130`	R1 = R0 (`2E` = `2E`), so jump to `30`
30	`C000`	halt

Answers: R2 = `0C`, RA = `22`, R0 = `2E`, **6 instructions**. If `1E` had been `80A2` (AND instead of OR): R0 = `22` ∧ `0C` = `00` ≠ `2E`, the jump is *not* taken and execution continues at `24`, `A248`, where the rotate has a non-zero third digit and is not a valid instruction; so the count cannot be determined (and `BCDE` would compare the uninitialised RC).

Key erratum. The old key's reasoning for Q2.5 is right; it just words the opcode problem loosely ("op-code A248 is not defined": the opcode `A` is defined, the operand format is wrong).

Test 2 §4.4A: program at `00`: `22FF`, `230A`, `5423`, `34AA`, `C000`. R2 = `FF` (−1), R3 = `0A` (10), R4 = R2+R3 = `09`, store R4 in cell `AA`, halt. So it computes $-1 + 10 = 9$ and stores it in `AA`. After the run **PC** = `0A` (HALT at `08`, plus 2) and **IR** = `C000`.

2.1 Writing programs

Method: (1) decide which register holds what, (2) load constants with `2RXY`, (3) load/store memory with `1RXY`/`3RXY`, (4) finish with `C000`, (5) add a comment per line (the exam asks for them). Work out addresses by counting 2 per instruction.

Add 3 to the value in `B0` and store the result in `B1` (checked in the simulator: `07` → `0A`):

```
00  10B0  R0 ← mem[B0]
02  2103  R1 ← 03
04  5201  R2 ← R0 + R1
06  32B1  mem[B1] ← R2
08  C000  halt
```

A loop with a jump that counts 1 up to `0A` in cell `AA` (Test 2 §4.4B), checked in the Vole simulator (final `AA` = `0A`):

```
00  2001  R0 ← 01   (counter)
02  210A  R1 ← 0A   (limit)
04  2201  R2 ← 01   (step)
06  30AA  mem[AA] ← R0   (loop start:  store counter)
08  B10E  if R0 = R1 (counter = limit) jump to 0E
0A  5002  R0 ← R0 + R2   (count up)
0C  B006  jump to 06   (R0 = R0, always)
0E  C000  halt
```

The jump instruction only tests *equality with R0*: that is why the counter lives in R0 and the exit test is "R1 = R0", and an unconditional jump is `B0XY`.

3 Operating systems and networks (Brookshear ch. 3 and 4)

3.1 The 20–50 word answers

These are the ones the old exams asked, plus the likely siblings. The wording below fits in 20–50 words; the key's own phrasing is marked (key).

Application vs. system software (key). System software provides the infrastructure that application software needs: the operating system and utilities. Application software performs the specific tasks the user wants, e.g. word processors, spreadsheets, databases.

Bootstrap (key). When the machine is switched on, a small boot loader program in ROM runs, loads the operating system from mass storage into main memory and then jumps to it. The operating system takes over from there.

Device driver. Software component of the operating system that knows the details of one particular peripheral device and translates general requests into that device's commands (Test 1 §3.1: the answer is "device drivers", not the file or memory manager).

Interrupt and time slice. A process runs for a fixed **time slice**; the end of it is signalled by an **interrupt** (Test 1 §3.2). The CPU saves the state, and the scheduler picks the next process.

Process vs. program. A program is a static set of instructions; a process is the *activity* of executing it: program plus current state (PC, registers, memory).

Context switch. Saving the state of the running process and loading another's, so time-sharing multitasking can alternate between processes. It costs time but no useful work.

Mutual exclusion / critical region. Only one process at a time may use shared data; the instructions that touch it form a critical region. Without it, interleaved updates give wrong results.

Test-and-set. A single indivisible instruction that tests a flag and sets it; since an interrupt cannot split it, it makes locks safe to implement.

Deadlock. Processes wait for each other's resources forever. Requires: competition for non-shareable resources, resources requested piecemeal, and no forced retrieval of an allocated resource.

Telnet vs. SSH (key). SSH encrypts the communication so eavesdroppers cannot read passwords or other sensitive data; Telnet sends everything in clear text.

Round-robin arithmetic. 3 processes, 20 ms slices, 10 µs per context switch: one round $= 3 \cdot 20\,\text{ms} + 3 \cdot 0.010\,\text{ms} = 60.03\,\text{ms}$, so in 1 s a process gets $1000/60.03 \approx 16.7$ turns (16 complete rounds). If B yields after 5 ms the round is $20 + 5 + 20 + 0.03 = 45.03\,\text{ms}$ and $1000/45.03 \approx 22.2$ turns. *Method: round time = sum of slices + number of switches × switch time; rounds per second* $= 1000/$*round.*

3.2 Networks cheat sheet

- **TCP/IP has four layers**, top to bottom: **application** (HTTP, SMTP, FTP, Telnet, SSH, DNS lookups), **transport** (**TCP**, **UDP**: segments, sequence numbers, ports), **network** (**IP**: forwarding and routing), **link** (Ethernet, WiFi). Test 1 §3.5: "transport and network layer" = **TCP, UDP and IP**. OSI has seven layers; routers only implement network and link.
- **Who may transmit (Test 2 §1.2). CSMA/CD** (Ethernet bus): listen, send, if a collision is detected stop and retry after a random delay. **CSMA/CA** (WiFi, 802.11): collisions cannot be detected over radio (your own signal drowns the others; hidden terminals), so wait and avoid collisions, often with a request/acknowledge handshake with the access point. *Wireless star networks → CSMA/CA.*
- **TCP vs. UDP:** TCP sets up a connection, is reliable (acknowledgements, retransmission) and controls flow; UDP is connectionless, unreliable but efficient (DNS, streaming).

- Bus vs. star is about logical topology; WiFi is a star around the access point; Ethernet is a bus. Repeater, bridge, switch connect parts of *one* network; a router joins *different* networks (an internet). Client/server vs. peer-to-peer describes how *processes* talk.

(Details: book sections 4.1–4.5.)

4 Algorithms and Python (Brookshear ch. 5, practice in Python)

4.1 Tracing loops

Method: "what is printed?"

Make a table with one column per variable and one row per *test of the condition*. Write down what is printed *when* it is printed (before or after the update). Check the condition with the exact operator ($<$ vs. $\leq$, $\neq$) before every pass.

Test 1 §4.1: `X=5; while X<7: print X; X=X+1` prints 5, 6 and leaves $X = 7$; `print(X)` prints 7; then `while X>2: print X; X=X-2` prints 7, 5, 3 (X becomes 1). Output: **5, 6, 7, 7, 5, 3**. Another: `x=3; while x<12: print x; x+=4` prints 3, 7, 11 and leaves 15; `print(15); while x>0: print x; x-=5` prints 15, 10, 5 (x then 0): **3 7 11 15 15 10 5**.

Equal sequences? (Test 2 §2.2). Trace each: A prints 5; B prints 5 (the print comes *after* the increment); C (`while True` with a `break` when $X > 6$) prints 5, 6. So C differs. The differences are always at the boundary: $<$ vs. $\leq$, a `break` test that runs after the update, or the position of the update relative to the print.

Flaw in loop control (Test 1 §4.3): `X=3; while X != 8: X=X+2`. X takes only odd values $3, 5, 7, 9, \ldots$ and never equals 8: the termination condition is never met (infinite loop). Use $<$ or $\leq$ instead of $\neq$ when the step might jump over the target.

Which statement needs no Boolean condition? (Test 1 §5.1) If-else, while and for need one; an *assignment* does not. Test 2 §4.1: "which is not a loop": if-else.

Count-controlled (`for`: repeats a known number of times) vs. **condition-controlled** (`while`: repeats while a condition holds, number of passes unknown).

4.2 Search

- **Sequential search** checks entries from the start; **binary search** (sorted list only) tests the middle entry and discards half. For an *even*-length list the course's "middle" is the **first entry of the second half**. Worst case $\lfloor \log_2 n \rfloor + 1$ tests, e.g. 200 entries $\to$ 8, 100 000 entries $\to$ 17 (book §5.5 Q2).
- **Test 1 §4.2:** Kelly is the 2nd of 11 names, so **sequential** finds it faster (2 tests vs. 3).
- **Test 2 §2.3B:** binary search for Kelly in John, Kelly, Lewis, Maurice, Nathan, Oliver, Pat, Quincy, Roger, Stan, Tom (11): test **Oliver** (middle), Kelly $<$ Oliver so the first half, John…Nathan (5): test **Lewis**, Kelly $<$ Lewis, so John, Kelly (2, even: the middle is the second entry): test **Kelly**: found. Names examined: Oliver, Lewis, Kelly; that is **2 recursive calls** (3 calls to Search in total, counting the first).
- **Seven names** Anna, Brita, Dóra, Elin, Hans, Ivar, Karl, target Karl: Elin (4th), then Hans–Karl: Ivar, then Karl: Elin, Ivar, Karl.

4.3 Writing the loops (templates)

```
# sum, product, count, min and max of an array
x = [5, 7, 2, 4, 48, 27, 2, 9, 3, 6]
total = 0
for v in x:             # or: i = 0; while i < len(x): ... i = i + 1
    total = total + v
print(total)

def sum_and_average(X):          # Test 2 S2.4C
    total = 0
    count = 0
    for v in X:
        total = total + v
        count = count + 1
    print(total, total / count)
```

```
15
16 maximum = 0                          # Test 1 S4.4: values are 0..999
17 minimum = 999
18 index = 0
19 while index < 10:
20     if x[index] > maximum: maximum = x[index]
21     if x[index] < minimum: minimum = x[index]
22     index = index + 1
23 print(maximum)
24 print(minimum)
25
26 count = 0                            # how many entries are even
27 for v in x:
28     if v % 2 == 0: count += 1
29 print(count)
30
31 def largest(X):                      # start from the first entry, not from 0
32     best = X[0]
33     for v in X:
34         if v > best: best = v
35     return best
```

Checklist: initialise before the loop (sum = 0, product = 1, min = above the largest possible, max = below the smallest); update inside; do not print inside the loop unless asked; a function *returns* or *prints* as the question says.

4.4 More Python from the labs (Moodle labs 5a, 5b, 6b and the array/function notes)

The teacher's labs show the style of "write this in Python" questions: `while` loops over an index, `%` for divisibility, and `ord`/`chr` for characters.

```
1  # even or odd, counting down (lab 5a)
2  x = int(input("Enter an integer: "))
3  while x > 0:
4      if x % 2 == 0: print(x, 'is even')
5      else:          print(x, 'is odd')
6      x = x - 1
7
8  # all factors of n: test every number 1..n with the remainder operator (lab
       5b)
9  factor = 1
10 while factor <= n:
11     if n % factor == 0: print(factor, end=" ")
12     factor = factor + 1
13
14 # Euclid's algorithm for the greatest common divisor (lab 5b, book 5.2 Q3)
15 while smaller != 0:
16     remainder = larger % smaller
17     larger = smaller
18     smaller = remainder
19 print(larger)                         # gcd(larger, smaller) before the loop
20
21 # Caesar cipher: shift each character code (lab 6b)
22 new_word = ''
23 for letter in word:
24     new_word = new_word + chr(ord(letter) + 4)
25
26 # arrays with while (array notes): reverse, count occurrences, longest name
27 reversed_list = [0] * len(numbers)
28 i = len(numbers) - 1; j = 0
```

```
while i >= 0:
    reversed_list[j] = numbers[i]; i = i - 1; j = j + 1
longest = names[0]; i = 1
while i < len(names):
    if len(names[i]) > len(longest): longest = names[i]
    i = i + 1
```

Reminders: `ord('A')` is 65, `chr(72)` is `'H'`; `//` is integer division and `%` the remainder; `bin(x)` and `hex(x)` give `0b...` and `0x...`; `x = [0] * 5` makes five zeros; indices start at 0, so the last index is `len(x)-1`. A function *with* `return` is a fruitful function; one that only prints is a void function. Bitwise in Python: `0b10011010 & 0b11001001` = `0b10001000`, `|` gives `0b11011011`, `^` gives `0b1010011`.

Insertion sort (book Fig. 5.11) traced on Fred, Alex, Diana, Byron, Carol: the pivot is the Nth entry, larger entries above it slide down, the pivot drops into the hole. After $N = 2$: Alex, Fred, Diana, Byron, Carol; $N = 3$: Alex, Diana, Fred, Byron, Carol; $N = 4$: Alex, Byron, Diana, Fred, Carol; $N = 5$: Alex, Byron, Carol, Diana, Fred.

4.5 Recursion

Method: write the calls as a stack, note what happens *before* and *after* the recursive call. Example: `def f(n): if n == 0: return 1; print(n); return n * f(n - 1); f(3)` prints $3, 2, 1$ (the print is before the recursive call) and returns $3 \cdot 2 \cdot 1 \cdot 1 = 6$. A `for`-loop equivalent is `for i in range(n, 0, -1): print(i); result *= i`. Book §5.5 Q3: `Exercise(N)` prints `N`, recurses while $N < 3$, then prints `N` again: $1, 2, 3, 3, 2, 1$ (prints on the way down *and* back up).

5 Programming languages (Brookshear ch. 6)

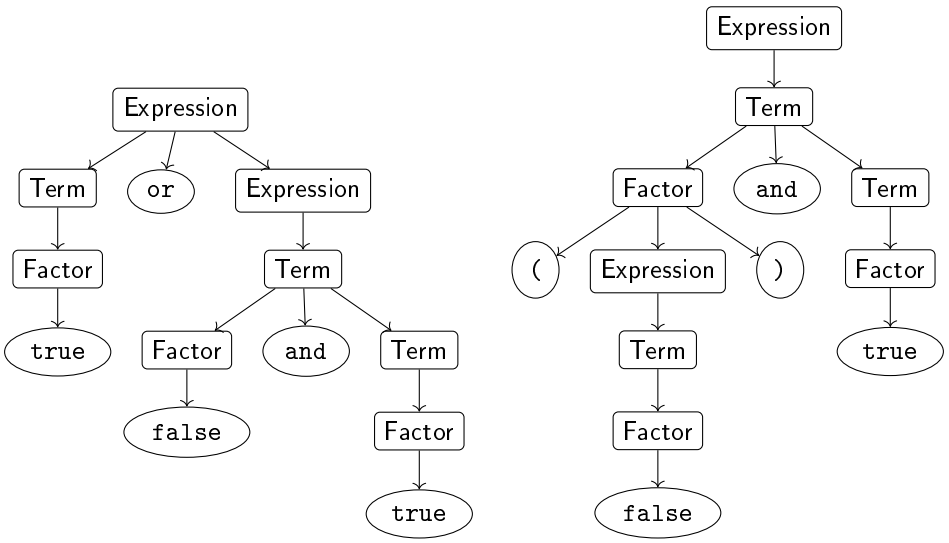
- **Translating a program** converts it to another (usually lower-level) language *without running it*; **interpreting** executes it directly from the high-level form (key).
- **Translation phases:** lexical analysis (scanner: characters → tokens), parsing (tokens → parse tree according to the grammar), code generation. *Executing the program is not a translation step* (Test 1 §5.2: answer A).
- **Disadvantages of machine/assembly language that high-level languages overcome (key):** they are machine dependent (not portable) and force algorithms into tiny machine-oriented steps instead of larger application-oriented ones.
- **Global vs. local variable (key):** a global variable is accessible throughout the program; a local one only within a specific region (e.g. the procedure that declares it). **Scope** = where it can be accessed (Test 2 §5.2A: "the portion of the program in which the variable can be accessed"); **lifetime** of a local = only while its procedure runs.
- **By value vs. by reference:** by value passes a *copy*, so changes inside do not affect the caller; by reference passes the address, so the function can change the original.

5.1 Syntax diagrams and parse trees

Method: draw a parse tree from the grammar

Each box in a syntax diagram is a **non-terminal** (a node with children), each circle is a **terminal** (a leaf: a literal word or symbol). Start with the start symbol; for the first branch of its diagram that matches the sentence, write the children left to right; expand each non-terminal the same way; the leaves, read left to right, must equal the sentence.

With **Expression** → Term [`or` Expression], **Term** → Factor [`and` Term], **Factor** → `(` Expression `)` | `true` | `false`:



(`true or false and true` on the left, `(false) and true` on the right.) In the left tree the `and` binds tighter than the `or` because Term sits below Expression. A parenthesised factor always contains a full Expression, which is where the recursion comes from.

6 Software engineering and UML class diagrams (Brookshear ch. 7)

- **Life cycle phases:** analysis, design, implementation, testing (then maintenance).
- **Methodologies: waterfall** is the most rigid (each phase finished before the next; Test 1 §6.1); **incremental** (grow in pieces), **evolutionary prototyping** (refine a prototype), **extreme / agile** (short iterations, constant feedback).
- **Modularity: cohesion** (a module does one thing; high is good) and **coupling** (dependence between modules; low is good). Top-down: refine from the whole into parts; bottom-up: build from parts.

6.1 Class diagrams and multiplicity

Reading and writing multiplicities

A number at an end of a line says *how many objects of that class* are attached to one object at the other end. `1` exactly one, `0..1` optional, `*` or `0..*` zero or many, `1..*` one or more. **"A has many B, each B belongs to one A"** ⇒ `A 1 -- * B`: **one-to-many**. Both ends many: many-to-many. A **composition** (filled diamond at the owner) means the part cannot exist without the whole; an **aggregation** (hollow diamond) is weaker.

- **Test 1 §6.3:** Customer `*` — `*` Account `*` — `1` Bank. Customer–Account is **many-to-many** (a customer can have many accounts, an account can have many customers); Bank–Account is **one-to-many** (a bank has many accounts, each account belongs to one bank).
- **One-to-many examples (Test 1 §6.2):** classroom → chairs, mother → children, department → employees.
- **Test 2 §4.3 (Library–Bookshelf–Book):**



Short description to write: a library has one or more bookshelves and each bookshelf belongs to exactly one library; a bookshelf holds zero or more books and each book is on one bookshelf. Both are one-to-many; they are compositions if a shelf cannot exist without its library and a book (copy) without its shelf. State the multiplicities; the description is only the sentence form of them.

7 Data structures in memory (Brookshear ch. 8)

7.1 Linked lists in memory

Each entry is two cells: `data`, `pointer to next`; null pointer = `00`; the **head pointer** is a separate value giving the address of the first entry. Follow the pointers from the head, *not* the address order.

Method: insert or replace in a linked list

Insert X after P: set the new entry's pointer to what P pointed to, *then* set P's pointer to the new entry. Delete: let the previous entry point past it. Replace the k-th entry: new entry's pointer = the old one's *next*, previous entry's pointer = the new entry.

Test 1 §7.3 (head = 52; entries 52:BB→58, 58:EE→54, 54:CC→00; the new data is at 50:AA). The second entry is EE at 58. The new entry must point to what EE pointed to and the first entry must point to the new one: **cell 51 ← 54** and **cell 53 ← 50**.

Sorted list example: data cells 70:05, 72:01, 74:09, 76:03 with empty pointer cells at 71, 73, 75, 77. Ascending order is 01 (72) → 03 (76) → 05 (70) → 09 (74) → 00, so pointers 73 = 76, 77 = 70, 71 = 74, 75 = 00 and the head pointer is 72.

7.2 Binary trees in memory

Each node is three cells: `data`, `left pointer`, `right pointer` (null = `00`). **Method:** start at the root address; read the three cells; for each non-null pointer go to that address and repeat; draw left child to the lower left, right child to the lower right.

- **Test 1 §7.1:** root at 50: data A, left pointer (51) = 56, right pointer (52) = 53. Node 56 has data C, node 53 has data B, all their pointers 00. So A has left child C and right child B.
- **Test 2 §4.2** (root at 5C): 5C: data 01, left 50, right 53. Node 50: data 09, leaf. Node 53: data 05, left 59, right 00. Node 59: data 02, leaf. Tree: 01 with left child 09 and right child 05, and 05 has left child 02. To insert the node at 56 as the right child of the node with value 02 (at 59): change the right pointer of that node, **cell 5B, from 00 to 56**.

The **pointer to change is always the left or right pointer cell of the parent**: data cell +1 is left, data cell +2 is right.

7.3 Tree vocabulary and traversals

Root, parent, child, sibling, leaf, subtree; a **binary** tree has at most 2 children per node; a **binary search tree** (BST) has, for every node, everything in its left subtree smaller and everything in its right subtree larger (so a tree is *not* a BST as soon as one value sits in the wrong subtree, e.g. a smaller value in the *right* subtree of its ancestor).

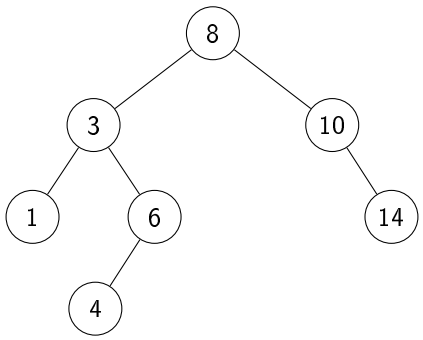
Traversal orders (Test 1 §7.2, Test 2 §3.1F)

in-order (Left, Value, Right):	prints a BST in sorted order
pre-order (Value, Left, Right):	root first
post-order (Left, Right, Value):	root last
reverse in-order (Right, Value, Left):	sorted in descending order
Value, Right only:	the root, then follows the right children down

Read the *code*: the order of the `print` relative to the two recursive calls is the whole answer.

Example: tree K (G (C, H), T (–, W)): reverse in-order → W, T, K, H, G, C; in-order → C, G, H, K, T, W; post-order → C, H, G, W, T, K. The sibling of G is T; the only node with one child is T; it is a binary tree.

Building a BST: insert 8, 3, 10, 1, 6, 14, 4: 8 is the root; each new number goes left if smaller, right if larger, repeating at every node. Result:



In-order gives 1, 3, 4, 6, 8, 10, 14.

7.4 Stacks, queues and how structures are stored (book ch. 8, slides)

- **List:** entries in sequence, with a **head** (start) and **tail** (end). **Stack:** insert and remove only at the head (the **top**); **push**/**pop**; **LIFO**. Used for backtracking and for recursive calls (the activations are pushed). **Queue:** remove at the head, insert at the tail; **FIFO**; used for buffers and the job queue.
- **Book §8.1 Q3:** push A, B, C; pop (C comes off); push D, E: stack top to bottom is **E, D, B, A**, and the next pop gives E. **Q4:** queue A, B, C; remove A; insert D, E: head to tail is **B, C, D, E**, and the next removal is B. **Q5:** A and C siblings, D child of A: the root is the (unnamed) parent B, the leaves are C and D.
- **Array** (same-type entries, index), **aggregate** (fields of different types, accessed by name; e.g. Employee with Name, Age, SkillRating). A Python list is a flexible array.
- **Static vs. dynamic:** a static structure keeps its size and shape; a dynamic one grows and shrinks (needs memory management and **garbage collection**; failing to reclaim space is a **memory leak**).
- **Storing:** an array is stored contiguously; in **row-major order** element $[i][j]$ of an array with C columns (0-based, cells of size s, start x) is at $x+(i\cdot C+j)s$ (the "address polynomial"). Lists are **contiguous** or **linked** (head pointer, null at the end). Stacks are normally stored contiguously; queues as a **circular queue** (the first cell follows the last, so the queue does not crawl out of its storage).
- **Binary tree without pointers:** `A[1]` is the root, the children of `A[i]` are `A[2i]` and `A[2i+1]` (a sparse, unbalanced tree wastes cells); with pointers: node = data + left + right.
- **Abstract data type (ADT):** data plus the functions that manipulate it (a stack with `push`, `pop`, `isEmpty`, `isFull`); users see the abstraction, not the memory. A **user-defined type** uses an aggregate (`struct`) or a class.

Binary search tree in code (slides 8.21–8.26):

```
def Search(Tree, TargetValue):
    if Tree is None:                        return None          # search failed
    elif TargetValue == Tree.Value:         return Tree
    elif TargetValue < Tree.Value:          return Search(Tree.Left,
       TargetValue)
    else:                                   return Search(Tree.Right,
       TargetValue)

def Insert(Tree, NewValue):
    if Tree is None:                        # empty spot: create a new leaf
        Tree = TreeNode(); Tree.Value = NewValue
    elif NewValue < Tree.Value:             Tree.Left  = Insert(Tree.Left,
       NewValue)
    elif NewValue > Tree.Value:             Tree.Right = Insert(Tree.Right,
       NewValue)
    return Tree

```

```
14 def PrintList(List):                    # linked list: follow Next until
       None
15     CurrentPointer = List.Head
16     while CurrentPointer is not None:
17         print(CurrentPointer.Value)
18         CurrentPointer = CurrentPointer.Next
```

Search and insert both walk one root-to-leaf path: left if smaller, right if larger; a duplicate changes nothing.

7.5 Arrays

A 2-D array stored in **row-major order** is stored row after row, so `Y[1][3]` (row 1) appears *before* `Y[3][1]` (row 3) (Test 2 §3.2: answer A). Column-major would be the opposite.

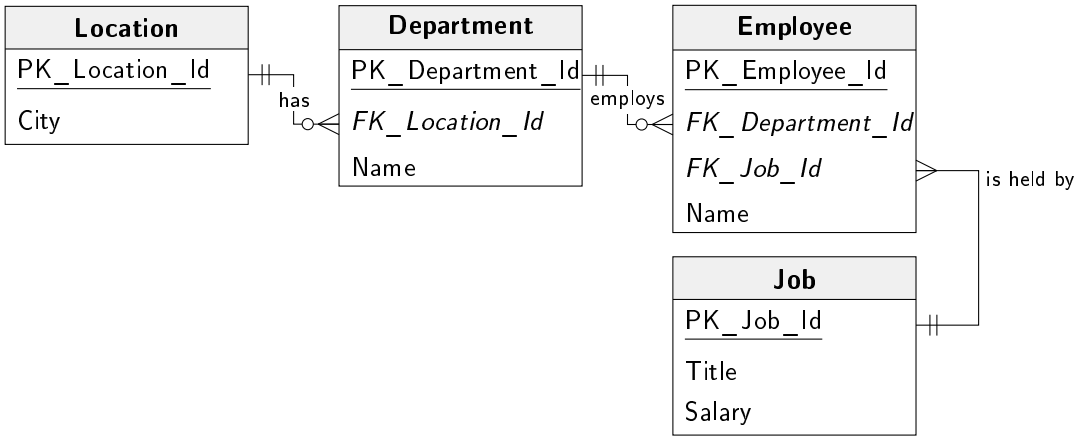
8 Databases (Brookshear ch. 9 and the ER notes)

8.1 From tables to an ER diagram

Method: crow's foot from PK/FK

Every **FK_x** column in table T is a relation to the table whose **PK** it refers to. Draw a line between them with a **single bar (exactly one)** at the PK table and a **crow's foot with a circle (zero or many)** at the table holding the FK: **one row in the PK table relates to many rows in the FK table**. Name each line with a verb and say it in words from both sides.

Test 1 §8.2 (Employee: PK, FK_Department_Id, FK_Job_Id, Name; Department: PK, FK_Location_Id, Name; Location: PK, City; Job: PK, Title, Salary):



Explanation to write: a location has many departments, each department is in one location; a department has many employees, each employee works in one department; a job can be held by many employees, each employee has one job. (The same pattern with a *linking table*: customers and stores with an order table holding both FKs, or members and books with a loan table holding both FKs. A **many-to-many** between two entities is always implemented with a linking table holding two FKs.)

8.2 Database vocabulary (book 9.1–9.2 and the ER notes)

- **Database:** data with internal links, so it is reachable from several perspectives (unlike a flat file). **DBMS:** the software layer between applications and the data. **Schema:** the structure of the whole database; **subschema:** the part relevant to one user (hides sensitive data). **Data independence:** the organisation can change without changing the application software.
- **Relational model:** a **relation** is a table; an **attribute** is a column; a **tuple** is a row (an instance of the entity). **Primary key** identifies a row; **foreign key** points to a row in another table.
- **Design:** avoid mixing several concepts in one relation (redundant data, and deleting a row can delete unrelated facts). **Decomposition** splits the columns into several relations, repeating the key columns needed to keep the link; it must be **lossless** (joining the parts gives back exactly the original).
- **Relationship types in the ER diagram:** one-to-one (a plain line), one-to-many (a crow's foot on the many side), many-to-many (made of two one-to-many with a linking table). The crow's foot is on the table that holds the foreign key. In the ER notes, address is its own table: a school or a person *has* an address (FK to Address), so one address can be shared by several of them.
- **SQL also changes data:**

```
insert into Employee values ('43212', 'Sue A. Burt', '33 Fair St.',
    '444661111');
update Employee set Address = '1812 Napoleon Ave.' where Name = 'Joe E.
    Baker';
```

```
delete from Employee where Name = 'G. Jerry Smith';
```

8.3 Relational algebra (Brookshear's notation)

SELECT picks *rows* that satisfy a condition; **PROJECT** picks *columns*; **JOIN** combines two relations row by row where a condition holds (usually `FK = PK`). Write one operation per line: `NEW` ← ..., ending with `RESULT`.

Test 1 §8.1. $X(U,V,W) = \{(1,A,E),(2,B,B),(3,C,C)\}$, $Y(R,S) = \{(3,Z),(2,A),(3,B)\}$:

operation	result
`PROJECT W from X`	$E,\ B,\ C$
`SELECT from X where U=3`	$(3,C,C)$
`JOIN X and Y where X.U = Y.R`	$(2,B,B,2,A),\ (3,C,C,3,Z),\ (3,C,C,3,B)$
`JOIN X and Y where X.U > Y.R`	$(3,C,C,2,A)$ (only $3>2$)

Typical sequences (Test 1 §8.3):

```
RESULT ← PROJECT Name from Employee
NEW1   ← JOIN Employee and Job where Employee.FK_Job_Id = Job.PK_Job_Id
RESULT ← PROJECT Name, Title from NEW1
NEW1   ← JOIN Employee and Job where Employee.FK_Job_Id = Job.PK_Job_Id
NEW2   ← SELECT from NEW1 where Salary > 1000
RESULT ← PROJECT Name from NEW2
```

Key erratum. Test 1 §8.3(2) in the key projects only `Title`, but the question asks for job titles *and employee names*: project both (as above).

8.4 SQL

```
select Name from Employee;                                          -- 8.4 (1)
select Employee.Name, Job.Title                                     -- 8.4 (2)
from Employee join Job on Employee.FK_Job_Id = Job.PK_Job_Id;
select Location.City, Employee.Name                                 -- 8.4 (3)
from Employee
join Department on Employee.FK_Department_Id = Department.PK_Department_Id
join Location   on Department.FK_Location_Id  = Location.PK_Location_Id;
select Employee.Name, Job.Title, Department.Name                    -- 8.4 (4)
from Employee
join Department on Employee.FK_Department_Id = Department.PK_Department_Id
join Location   on Department.FK_Location_Id  = Location.PK_Location_Id
join Job        on Employee.FK_Job_Id         = Job.PK_Job_Id
where Location.City = 'Torshavn' and Job.Salary > 1000;
select distinct Title from Job where Salary > 1000;                 -- distinct removes repeats
```

Pattern: `select` the columns, `from` the first table, one `join ...on FK = PK` per extra table, `where` the conditions; `distinct` removes repeats. The **number of joins is one less than the number of tables used**. Qualify column names with the table name when two tables share a name (`Name`!).

Key erratum. Key §8.4(2) has `from Employee, Title`; the table is `Job`, not `Title`. The key also uses the old comma style (`from A, B where ...`), which is equivalent to `join`.

Test 2 §5.1 (persons, events, participation): the participation table holds the two FKs; "persons living in Denmark who took part in the event ROYAL RUN in 2019" joins person → participation → event, with `where Person.Country='Denmark' and Event.Name='ROYAL RUN'` and a condition on the year of the event's start time ("`START_TIME` includes the year": a comparison such as `START_TIME >= '2019-01-01' and START_TIME < '2020-01-01'`).

9 The rest of the syllabus (slides and book, not in the old exams)

The old exams are only a sample. The slides on Moodle for chapters 1, 2, 3, 5, 6, 8 and 9 stress the following points, so any of them can turn up as a "list / explain / short answer" question. (No slides for chapters 4 and 7 were on Moodle on 13 Oct; those parts come from the book.)

9.1 Data storage (ch. 1)

- **Boolean operations** AND, OR, XOR, NOT; a **gate** computes one; a **flip-flop** (built from gates) stores one bit: one input sets it to 1, another sets it to 0, and with both inputs 0 the last value is kept. This is the basis of memory.
- **Main memory:** cells (usually 8 bits = 1 byte), each with a numeric **address** starting at 0; **most significant bit** is the left end. **RAM** (any cell reachable in any order), **DRAM** (volatile). KB $= 2^{10}$, MB $= 2^{20}$, GB $= 2^{30}$ bytes.
- **Mass storage:** magnetic disk, tape, CD/DVD/BD, flash and SSD. Compared with main memory: less volatile, larger, cheaper, often removable. **Bandwidth** = bits transferred per second; **latency** = time from request to arrival. Flash wears out from repeated erasing.
- **Text: ASCII** 7 bits per symbol; ISO 8-bit extensions per language group; **Unicode** up to 21 bits (UTF-8 for English text is ASCII-compatible).
- **Images: bit map** (pixels; **RGB** = three components per pixel; poor scaling) versus **vector** (geometric shapes; scalable; TrueType, PostScript). **Sound: sampling** (8 000 samples/s telephone, 44 100 samples/s CD) versus **MIDI** (instructions: instrument, note, duration).
- **Compression: lossless** (exactly recoverable) vs. **lossy**. Techniques: **run-length** (value + repeat count), **frequency-dependent** (Huffman: frequent symbols get short codes), **relative/differential** (store differences), **dictionary** (LZW: grows a dictionary of seen strings). Formats: **GIF** (cartoons), **JPEG** (photos), **TIFF** (archiving), **MP3**, **MPEG**.
- **Round-off:** a mantissa that is too short *truncates* the value; $\frac{1}{10}$ has no exact binary expansion. **Numerical analysis:** the order of adding values changes the result; adding a tiny value to a huge one loses it.
- **Python error kinds (1.8): syntax** error (`print(5 +)`, `pront(5)` is a name error), **semantic** error (wrong expression, e.g. `40 + extra * rate`), **runtime** error (divide by zero).

9.2 CPU, machine cycle and I/O (ch. 2)

- **CPU: arithmetic/logic unit**, **control unit**, **register unit** (general-purpose and special-purpose registers); connected to main memory by the **bus**.
- **Stored-program concept:** the program is bit patterns in main memory, so changing the program means changing memory, not rewiring the CPU.
- **Machine cycle: fetch** (read the instruction at the PC into the IR, PC advances), **decode**, **execute**; repeat. The two special registers: **instruction register** (current instruction), **program counter** (address of the next one).
- **Instruction parts: op-code** (which operation) and **operands** (details). **Types:** data transfer (LOAD, STORE), arithmetic/logic (add, AND, OR, XOR, rotate, shift), control (JUMP/BRANCH).
- **RISC** (few simple fast instructions; ARM, PowerPC) vs. **CISC** (many powerful instructions; Intel).
- **Masking with logic:** AND with 0 clears a bit and with 1 keeps it (`AND 00001111` keeps the low nibble); OR with 1 sets a bit; XOR with 1 flips it. **Rotate** wraps bits around; **logical shift** fills with 0; **arithmetic shift** keeps the sign bit.
- **Devices:** a **controller** handles communication with a device (specific or general like USB, HDMI); a **port** is where it connects. **Memory-mapped I/O:** devices appear as memory cells. **DMA:** a controller reads/writes main memory over the bus without the CPU; the **von Neumann bottleneck** is the CPU and controllers competing for the bus. **Handshaking** coordinates a transfer. **Parallel** (many lines at once, internal bus) vs. **serial** (one line, bit after bit, USB). Rates in bps, Kbps, Mbps, Gbps (powers of 1000); **bandwidth** is the maximum rate.

- A single Python statement can correspond to one, tens or hundreds of machine instructions; the programmer need not know whether the CPU is RISC or CISC.

9.3 Operating systems (ch. 3)

- **History: batch** processing (jobs in a **job queue**), **interactive**, **time-sharing** (one machine, many users), **multitasking** (one user, many tasks), **multiprocessor**, **embedded**. Examples: Windows, UNIX, macOS, Linux, Android, iOS.
- **Components: user interface** (text **shell** or **GUI**) and the **kernel**: **file manager** (directories, paths), **device drivers**, **memory manager** (allocates memory; **virtual memory** shifts blocks called **pages** between main memory and mass storage), **scheduler and dispatcher**.
- **Process** = the *activity* of executing a program; its **process state** is the PC, the registers and the related memory. The **scheduler** adds and removes processes in the process table; the **dispatcher** hands out time slices (multiprogramming); a slice ends with an **interrupt**.
- **Competition: critical region**, **mutual exclusion**, **semaphore** (a control flag guarding the region), **deadlock** (three conditions: non-shareable resources, requested piecemeal, no forced retrieval).

9.4 Programming languages and implementation (ch. 6)

- **Generations:** 1st **machine language** (e.g. Vole `156C`); 2nd **assembly language** (mnemonics and names for memory cells, *one-to-one* with machine instructions, machine dependent, converted by an **assembler**: `LD R5, Price`); 3rd **high-level** (FORTRAN, COBOL, C, Python; each primitive becomes several machine instructions; mostly machine independent; converted by a **compiler**).
- **Concepts:** data types (integer, real/float, character, Boolean), variables, constants and literals, data structures (array, aggregate), assignment, control (if, while, for, switch), comments (ignored by the translator). **Function** = subprogram, subroutine, procedure, method; **formal** parameters are in the definition, **actual** parameters (arguments) in the call; **fruitful** functions return a value, **void** ones do not.
- **Implementation:** the **lexical analyzer** finds tokens, the **parser** groups tokens into statements using the syntax diagrams (parse tree), the **code generator** produces machine instructions. A grammar is **ambiguous** if one sentence has two parse trees (the "dangling else": `if B1 then if B2 then S1 else S2`: does the `else` belong to the inner or the outer `if`?).

9.5 Algorithms: concepts (ch. 5)

- **Algorithm:** an ordered set of unambiguous, executable steps that defines a terminating process. The *algorithm* is abstract; a *program* is its representation (story vs. book); a *process* is the activity of executing it.
- **Pseudocode** is between natural language and a programming language (Python-like): assignment, `if`/`else`, `while`, functions with parameters; indentation shows nesting.
- **Pólya's four steps:** understand the problem; devise a plan; carry it out; evaluate the solution (and its use for other problems). Getting a foot in the door: work backwards, solve an easier related problem, **stepwise refinement** (= **top-down**) or solve pieces first (**bottom-up**).
- **Loops: pretest** (`while`: test before each pass, may run zero times) vs. **posttest** (`repeat ... until`: body runs at least once). Repetitive control has initialisation, test and modification. **Recursion** needs a **base case** (termination), a modification that moves towards it, and creates activations that wait for each other (only one runs at a time).

9.6 Software engineering: the rest of 7.1–7.5

- **Methodologies:** **waterfall** (analysis, design, implementation, testing, strictly in order, one direction); **incremental** (a simplified first version, then add features); **iterative** (refine each version); **prototyping** (**evolutionary** prototypes grow into the product, **throwaway**/**rapid** prototypes are discarded and used to clarify requirements); **open source**; **agile** methods, e.g. **extreme programming** (small team, shared workspace, short daily cycles of analysis-design-implementation-test, flexible). RUP is an iterative process.
- **Modularity:** coupling forms are **control coupling** (one module passes control to another, a function call), **data coupling** (modules share data: by parameters, or implicitly as **global data**, which should be used with caution) and cohesion forms **logical cohesion** (weak: parts grouped because they are similar) vs. **functional cohesion** (strong: one activity). Aim for **low coupling and high cohesion**. Object orientation tends to reduce data coupling.
- **Tools:** structure charts, data flow diagrams, **UML** (use case, class and sequence diagrams), **design patterns**, CASE tools, prototyping.

10 Chapter review problems from Moodle, worked (ch. 6–9)

The teacher listed these as the problems to do ("6.1" means Chapter Review Problem 1 of chapter 6, in the Global Edition). Problems that need a figure I cannot read from the text extraction are left out (7.32, 8.8, 8.30, 6.44, 6.36, 6.43). The Vole programs were run in the simulator.

Chapter 6: Programming languages

6.1 A language is **machine independent** if a program written in it can be translated to run on different machines without being rewritten.

6.2 `x = 0; while (x < 3): x = x + 1`. Vole has no "less than", but x steps by 1, so testing equality with 3 works. R0 $= x$, R1 $= 1$, R2 $= 3$:

```
00  2000  R0 ← 0
02  2101  R1 ← 1
04  2203  R2 ← 3
06  B20C  if R2 = R0 goto 0C
08  5001  R0 ← R0 + R1
0A  B006  goto 06
0C  C000  halt
```

(the simulator ends with R0 $= 3$).

6.3 `Halfway = Length + Width` with floating-point values; put Length at `A0`, Width at `A1`, Halfway at `A2`: `11A0` (R1 ← Length), `12A1` (R2 ← Width), `6012` (R0 ← R1 + R2 *floating point*, opcode 6, not 5), `30A2` (store), `C000`.

6.4 `if (X == 0): Z = Y + W else: Z = Y + X` (two's complement; X, Y, W, Z at `A0`, `A1`, `A2`, `A3`). A comparison is only possible against R0, so R0 is set to 0:

```
00  2000  R0 ← 0
02  11A0  R1 ← X
04  B10E  if X = 0 goto then (0E)
06  12A1  else:  R2 ← Y
08  5321  R3 ← Y + X
0A  33A3  Z ← R3
0C  B016  goto end
0E  12A1  then:  R2 ← Y
10  13A2  R3 ← W
12  5423  R4 ← Y + W
14  34A3  Z ← R4
16  C000  end:  halt
```

(tested: $X = 0, Y = 3, W = 4$ gives $Z = 7$; $X = 5$ gives $Z = 8$).

6.6 The three levels: **machine language**, **assembly language**, **high-level** (third generation) language.

6.13 A **declaration** states a name and its type (that the variable exists); the **definition** also reserves the storage (and may give an initial value).

6.14 A **local** variable is accessible only within the procedure (or block) that declares it and lives while that runs; a **global** variable is accessible everywhere in the program. Globals couple modules together invisibly, so locals are preferred.

6.15 Operator precedence says which operator is applied first when there are no parentheses. With the usual precedence $6 + 2 \cdot 3 = 12$; without it, left to right, $(6 + 2) \cdot 3 = 24$.

6.19 `x=1; while (x != 100): print(x); x = x + 1` → `for x in range(1, 100): print(x)` (prints 1 to 99).

6.21 `for (i=0; i<100; i++) { if (i % 2 == 0) printf(i) }` as a `while`: `i = 0; while (i < 100): if (i % 2 == 0): print(i); i = i + 1` (the increment must be inside the loop).

6.40 "yes^n no^n": Sentence → `yes` [Sentence] `no`, a diagram that contains itself (recursion); it generates "yes no", "yes yes no no", and so on.

6.42 **String** → `x` String `x` | `y`: "a string is either the single letter y, or a string with an x on each side". Parse tree of `xxyxx`: String → (x, String, x); that inner String → (x, String, x); the innermost String → y. So the leaves read x, x, y, x, x.

Further ch. 6 problems that are quick to answer: **6.7** f removes the letters at odd positions: "programming" → "rgamn" → "gm". **6.8** $f(f(a,b),g(a,b)) = (a+b)+(a-b) = 2a$. **6.23** the rat's nest means `if X > 5: X = X + 2 else: X = X + 1`. **6.26** an integer variable receiving 6.0 stores 6. **6.29** by value prints 7 then 5; by reference 7 then 7. **6.30** X global: by value $5, 9, 5$; by reference $9, 9, 9$; **6.31** value-result $5, 9, 9$ (the copy is written back only when the function ends). **6.33** `9/4 - 1` is ambiguous: integer division ($2 - 1 = 1$) or real division ($2.25 - 1 = 1.25$). **6.45** the generator can use the known value: in the `then` branch $X = 5$, so $Z = 7$ is a constant. **6.46** Y is known to be 5, so the test is false: just $Z = 9$. **6.47** `X = 5` (if $X \neq 5$ it becomes 5, otherwise it already is). **6.59** 0.01 has no exact binary form in the 8-bit float, so X never equals exactly 1.00 and the loop does not stop (truncation errors).

Chapter 7: Software engineering

7.1 Time spent on good requirements, design and documentation pays back in maintenance: a well-documented, modular system is cheaper to correct and extend (maintenance is the largest part of the life-cycle cost).

7.2 **Throwaway prototyping:** an incomplete version of the system is built quickly to clarify requirements or demonstrate it, then discarded and the real system is built afresh (as opposed to evolutionary prototyping, where the prototype grows into the product).

7.3 Without good **metrics** (measures of properties such as complexity) software engineering has to rely on judgement and experience; cost, quality and progress are hard to compare, estimate or guarantee, unlike in traditional engineering.

7.7 **Waterfall:** *disadvantage*, it is inflexible: changes in requirements are expensive and a working system appears only at the end. *Advantage*: orderly, with clear phases, milestones and documentation, suited to large well-understood projects.

7.8 **Open source** is closer to **bottom-up** / **incremental**: an initial version is written, then others modify and extend pieces of it and report back; there is no complete top-down design first.

7.9 A **constant** (named value) is changed in one place and carries a meaning (`DaysWk`); a literal repeated through the program must be found and changed everywhere, and one may be missed (cf. 6.34).

7.10 **Coupling** is the dependence between modules; **cohesion** is how closely a module's parts belong together. **Minimise** coupling (a change in one module should not ripple) and **maximise** cohesion (a module does one thing and is easy to understand and reuse).

7.13 **By reference** gives the more complex data coupling: the called function can change the caller's data, so the modules share the data; by value only a copy is passed.

7.20 Class diagram: `Publisher` 1 — $*$ `Magazine` (a publisher publishes many magazines, each magazine has one publisher) and `Magazine` $*$ — $*$ `Subscriber` (many-to-many: a subscriber takes many magazines and a magazine has many subscribers).

7.25 A **class diagram** shows the *static* structure: classes, their attributes and methods, and the associations (with multiplicity) between them. A **sequence diagram** shows the *dynamic* behaviour: which objects exchange which messages (method calls), in what order, over time.

Chapter 8: Data abstractions

8.2 6×8 array, row-major from address 50, rows and columns counted from 1: the entry in row 5, column 7 is number $(5-1)\cdot 8+(7-1)=38$ (counting from 0). 2 cells per entry: $50+38\cdot 2=$ **126**; 3 cells per entry: $50+38\cdot 3=$ **164**.

8.3 Column-major: number $(7-1)\cdot 6+(5-1)=40$. 2 cells: $50+80=$ **130**; 3 cells: $50+120=$ **170**.

8.5 A contiguous list has fixed size and gives direct access by index, which suits a **static** list. A **dynamic** list needs insertions and deletions, which mean shifting all following entries, and growth may need a bigger contiguous block; a linked list handles that by changing pointers.

8.7 Cells `11` (C), `13` (G), `15` (E), `17` (B), `19` (U), `1B` (F), each followed by an empty pointer cell. Alphabetical order is B, C, E, F, G, U, so the pointers are: `18` = `11` (B→C), `12` = `15` (C→E), `16` = `1B` (E→F), `1C` = `13` (F→G), `14` = `19` (G→U), `1A` = `00` (U last). **Head pointer** = `17`.

8.9 (the linked list at head 0x44) Following the pointers from `44`: J (pointer `4A`) → A (`40`) → N (`46`) → E (`00`): the name is **JANE**. To make it JEAN: J's pointer (`45`) = `46` (E), E's pointer (`47`) = `4A` (A), A's pointer (`4B`) = `40` (N), N's pointer (`41`) = `00`.

Chapter 9: Database systems

9.1 The **DBMS** is the software layer between applications and the stored data. It hides how and where data is stored (data independence), enforces access rights through subschemas, and keeps the data consistent when many users share it.

9.3 A **lossless (nonloss) decomposition** splits a relation into smaller ones, repeating the key columns, such that joining them again gives back exactly the original relation: no information lost and no invented tuples.

9.5 Two benefits of keeping application software separate from the DBMS: the **data organisation can change without changing the applications** (data independence), and the application programmer **need not write the storage, security and sharing code** (it is shared, and the same data serves many applications).

9.12 $A(L,M,N)=\{(X,P,1),(Y,Q,7),(Z,R,6)\}$, $B(X,Y)=\{(9,7),(4,3)\}$. (a) `SELECT from B where X = 9`: $(9,7)$. (b) `PROJECT L from A`: X, Y, Z. (c) `SELECT from A where L = Z`: $(Z,R,6)$. (d) `JOIN A and B where A.N = B.Y`: only $N=Y=7$ matches, giving $(Y,Q,7,9,7)$.

9.13 / 9.14 Course(CName, ID): (Networks, IT655), (Database, CS543), (VLSI, EC653); Branch(BName, ID, Credits). (a) "Which branches offer IT665?" (a misprint for IT655; there is no IT665):

```
NEW1   ← SELECT from Branch where ID = 'IT655'
RESULT ← PROJECT BName from NEW1
```

```
1 select distinct BName from Branch where ID = 'IT655';      -- Electronics &
    Communication, Information Technology
2 select distinct BName from Branch;                         -- (b) all
    branches: Computer Science, Electronics & Communication, Information
    Technology
3 select distinct BName from Branch where Credits = 4;       -- (c) branches
    with a 4-credit course: Computer Science, E&C, IT
```

(b) in relational algebra is `RESULT ← PROJECT BName from Branch`, (c) is `NEW1 ← SELECT from Branch where Credits = 4` followed by `RESULT ← PROJECT BName from NEW1`. (The course tables here need no join because `Branch` already holds the credits.)

11 Last-minute checklist

1. Two's complement: leading 1 → negative, invert +1; overflow only for same-sign operands.
2. 8-bit float: sign | exponent (excess-4) | .mantissa; value = $\pm.mmmm \times 2^{e-4}$.
3. Parity: count the 1s, compare with even/odd. Hamming: nearest code word.
4. Vole: `1`=load mem, `2`=load value, `3`=store, `5`=add, `B`=jump if =R0, `C`=halt. PC after halt = HALT address +2.
5. Trace loops in a table; watch $<$ vs. $\leq$ and print-before/after-update.
6. Binary search: "middle" of an even list = first entry of the second half.
7. Memory trees/lists: data, left, right / data, next; null = `00`; change the *parent's* pointer cell.
8. Traversals: read where the `print` is relative to the two recursive calls.
9. ER: FK sits on the *many* side. SQL: one `join` per extra table. Relational algebra: SELECT rows, PROJECT columns.
10. Short answers: 20–50 words, one definition and one contrast or example.